PLSQL EXERCISES

1.Control Structures

SCENERIO 1:

Code:

```
  DECLARE
  CURSOR cur_senior_loans IS
    SELECT c.CustomerID,
         c.CustomerName,
         c.Age,
         l.LoanID,
         l.InterestRate
    FROM   Customers c
    JOIN   Loans l ON c.CustomerID = l.CustomerID
    WHERE  c.Age > 60;

  v_new_rate  Loans.InterestRate%TYPE;
  v_count     NUMBER := 0;

BEGIN
  DBMS_OUTPUT.PUT_LINE('================================================');
  DBMS_OUTPUT.PUT_LINE('  Scenario 1: Senior Citizen Loan Discount      ');
  DBMS_OUTPUT.PUT_LINE('================================================');

  FOR rec IN cur_senior_loans LOOP

    v_new_rate := rec.InterestRate - 1;

    IF v_new_rate < 0 THEN
      v_new_rate := 0;
    END IF;

    UPDATE Loans
    SET    InterestRate = v_new_rate
    WHERE  LoanID = rec.LoanID;

    v_count := v_count + 1;

    DBMS_OUTPUT.PUT_LINE(
      'Customer : ' || rec.CustomerName  ||
      ' | Age: '   || rec.Age           ||
      ' | Loan ID: '|| rec.LoanID       ||
      ' | Old Rate: '|| rec.InterestRate || '%' ||
      ' -> New Rate: '|| v_new_rate      || '%'
    );

  END LOOP;

  COMMIT;
```

```
  DBMS_OUTPUT.PUT_LINE('-----------------------------------------------');
  DBMS_OUTPUT.PUT_LINE('Total loans discounted : ' || v_count);
  DBMS_OUTPUT.PUT_LINE('Status : Discount applied successfully.');
  DBMS_OUTPUT.PUT_LINE('===============================================');

EXCEPTION
  WHEN OTHERS THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);
END;
```

OUTPUT:

```
================================================
Scenario 1: Senior Citizen Loan Discount
================================================
Customer : Rajesh Kumar | Age: 65 | Loan ID: 101 | Old Rate: 12.5% -> New Rate:
11.5%
Customer : Anand Mehta | Age: 70 | Loan ID: 103 | Old Rate: 11% -> New Rate: 10%
Customer : Vikram Nair | Age: 62 | Loan ID: 105 | Old Rate: 13% -> New Rate: 12%
Customer : Mohan Das | Age: 68 | Loan ID: 107 | Old Rate: 12% -> New Rate: 11%
------------------------------------------------
Total loans discounted : 4
Status : Discount applied successfully.
================================================

PL/SQL procedure successfully completed.
```

SCENERIO 2:

CODE:

```
 DECLARE
  CURSOR cur_all_customers IS
    SELECT CustomerID,
        CustomerName,
        Balance,
        IsVIP
    FROM   Customers;

  v_vip_count     NUMBER := 0;
  v_non_vip_count NUMBER := 0;

BEGIN
  DBMS_OUTPUT.PUT_LINE('================================================');
  DBMS_OUTPUT.PUT_LINE('  Scenario 2: VIP Status Promotion              ');
  DBMS_OUTPUT.PUT_LINE('================================================');

  FOR rec IN cur_all_customers LOOP

    IF rec.Balance > 10000 THEN

      UPDATE Customers
      SET    IsVIP = 'TRUE'
      WHERE  CustomerID = rec.CustomerID;
```

```
        v_vip_count := v_vip_count + 1;

        DBMS_OUTPUT.PUT_LINE(
          '[VIP GRANTED] ' || rec.CustomerName ||
          ' | Balance: $' || rec.Balance      ||
          ' | Status: ' || rec.IsVIP || ' -> TRUE'
        );

      ELSE
        v_non_vip_count := v_non_vip_count + 1;

        DBMS_OUTPUT.PUT_LINE(
          '[NO CHANGE]   ' || rec.CustomerName ||
          ' | Balance: $' || rec.Balance      ||
          ' | Status remains: FALSE'
        );

      END IF;

    END LOOP;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('-----------------------------------------------');
    DBMS_OUTPUT.PUT_LINE('Customers promoted to VIP : ' || v_vip_count);
    DBMS_OUTPUT.PUT_LINE('Customers not eligible    : ' || v_non_vip_count);
    DBMS_OUTPUT.PUT_LINE('Status : VIP update completed successfully.');
    DBMS_OUTPUT.PUT_LINE('===============================================');

EXCEPTION
    WHEN OTHERS THEN
      ROLLBACK;
      DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);
END;
```

OUTPUT :

```
================================================
Scenario 2: VIP Status Promotion
================================================
[VIP GRANTED] Rajesh Kumar | Balance: $15000 | Status: FALSE -> TRUE
[NO CHANGE]   Priya Sharma | Balance: $8000 | Status remains: FALSE
[NO CHANGE]   Anand Mehta | Balance: $3000 | Status remains: FALSE
[VIP GRANTED] Sunita Verma | Balance: $12000 | Status: FALSE -> TRUE
[VIP GRANTED] Vikram Nair | Balance: $20000 | Status: FALSE -> TRUE
[NO CHANGE]   Deepa Iyer | Balance: $9500 | Status remains: FALSE
[NO CHANGE]   Mohan Das | Balance: $500 | Status remains: FALSE
------------------------------------------------
Customers promoted to VIP : 3
Customers not eligible    : 4
Status : VIP update completed successfully.
================================================

PL/SQL procedure successfully completed.
```

SCENERIO 3:

CODE:

```
 DECLARE
CURSOR cur_due_loans IS
  SELECT c.CustomerID,
       c.CustomerName,
       l.LoanID,
       l.InterestRate,
       l.DueDate,
       (l.DueDate - SYSDATE) AS DaysLeft
  FROM   Customers c
  JOIN   Loans l ON c.CustomerID = l.CustomerID
  WHERE  l.DueDate BETWEEN SYSDATE AND SYSDATE + 30
  ORDER  BY l.DueDate ASC;

v_reminder_count NUMBER := 0;

BEGIN
  DBMS_OUTPUT.PUT_LINE('================================================');
  DBMS_OUTPUT.PUT_LINE('  Scenario 3: Loan Due Date Reminders           ');
  DBMS_OUTPUT.PUT_LINE('  (Loans due within next 30 days)               ');
  DBMS_OUTPUT.PUT_LINE('================================================');

  FOR rec IN cur_due_loans LOOP

    v_reminder_count := v_reminder_count + 1;

    DECLARE
      v_urgency VARCHAR2(20);
    BEGIN
      IF rec.DaysLeft <= 5 THEN
        v_urgency := '[!! URGENT !!]';
      ELSIF rec.DaysLeft <= 15 THEN
        v_urgency := '[! DUE SOON !]';
      ELSE
        v_urgency := '[  REMINDER  ]';
      END IF;

      DBMS_OUTPUT.PUT_LINE(
        v_urgency                                  ||
        ' Dear ' || rec.CustomerName               ||
        ', your Loan (ID: ' || rec.LoanID || ')'   ||
        ' of ' || rec.InterestRate || '% interest' ||
        ' is due on ' || TO_CHAR(rec.DueDate, 'DD-MON-YYYY') ||
        ' (' || ROUND(rec.DaysLeft) || ' days left).'
      );
    END;

  END LOOP;

  DBMS_OUTPUT.PUT_LINE('------------------------------------------------');
```

```
  IF v_reminder_count = 0 THEN
    DBMS_OUTPUT.PUT_LINE('No loans due in the next 30 days.');
  ELSE
    DBMS_OUTPUT.PUT_LINE('Total reminders sent : ' || v_reminder_count);
  END IF;

  DBMS_OUTPUT.PUT_LINE('Status : Reminder process completed.');
  DBMS_OUTPUT.PUT_LINE('================================================');

EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);
END;
```

OUTPUT :

```
================================================
Scenario 3: Loan Due Date Reminders
(Loans due within next 30 days)
================================================
[!! URGENT !!] Dear Mohan Das, your Loan (ID: 107) of 11% interest is due on
25-JUN-2026 (3 days left).
[!! URGENT !!] Dear Anand Mehta, your Loan (ID: 103) of 10% interest is due on
27-JUN-2026 (5 days left).
[! DUE SOON !] Dear Rajesh Kumar, your Loan (ID: 101) of 11.5% interest is due
on 02-JUL-2026 (10 days left).
[! DUE SOON !] Dear Deepa Iyer, your Loan (ID: 106) of 10.75% interest is due on
07-JUL-2026 (15 days left).
[  REMINDER  ] Dear Sunita Verma, your Loan (ID: 104) of 9.5% interest is due on
17-JUL-2026 (25 days left).
------------------------------------------------
Total reminders sent : 5
Status : Reminder process completed.
================================================

PL/SQL procedure successfully completed.
```

3. Stored Procedures

SCENERIO 1:

CODE:

```
  CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest AS

  CURSOR cur_savings IS
    SELECT AccountID,
        CustomerName,
        Balance
    FROM   Accounts
    WHERE  AccountType = 'SAVINGS';

  v_interest      NUMBER(15, 2);
```

```
   v_new_balance   NUMBER(15, 2);
   v_total_interest NUMBER(15, 2) := 0;
   v_count        NUMBER := 0;
   v_interest_rate CONSTANT NUMBER := 0.01; -- 1% monthly interest

BEGIN

DBMS_OUTPUT.PUT_LINE('================================================
=');
   DBMS_OUTPUT.PUT_LINE('  Scenario 1: Monthly Interest Processing       ');
   DBMS_OUTPUT.PUT_LINE('  Interest Rate : 1% on Savings Accounts        ');

DBMS_OUTPUT.PUT_LINE('================================================
=');

   FOR rec IN cur_savings LOOP

      v_interest    := ROUND(rec.Balance * v_interest_rate, 2);
      v_new_balance := rec.Balance + v_interest;

      UPDATE Accounts
      SET    Balance = v_new_balance
      WHERE  AccountID = rec.AccountID;

      v_total_interest := v_total_interest + v_interest;
      v_count         := v_count + 1;

      DBMS_OUTPUT.PUT_LINE(
         'Account: ' || rec.AccountID       ||
         ' | Customer: ' || rec.CustomerName ||
         ' | Old Balance: Rs.' || rec.Balance ||
         ' | Interest: Rs.' || v_interest    ||
         ' | New Balance: Rs.' || v_new_balance
      );

   END LOOP;

   COMMIT;

   DBMS_OUTPUT.PUT_LINE('------------------------------------------------');
   DBMS_OUTPUT.PUT_LINE('Accounts processed     : ' || v_count);
   DBMS_OUTPUT.PUT_LINE('Total interest applied : Rs.' || v_total_interest);
   DBMS_OUTPUT.PUT_LINE('Status : Monthly interest processed successfully.');

DBMS_OUTPUT.PUT_LINE('================================================
=');
```

```
EXCEPTION
   WHEN OTHERS THEN
      ROLLBACK;
      DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);
END ProcessMonthlyInterest;
```

OUTPUT:

```
Procedure created.
```

SCENERIO 2:

CODE :

```
    CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus (
   p_department     IN Employees.Department%TYPE,
   p_bonus_percent  IN NUMBER
) AS

   CURSOR cur_dept_employees IS
      SELECT EmployeeID,
           EmployeeName,
           Salary
      FROM   Employees
      WHERE  Department = p_department;

   v_bonus        NUMBER(15, 2);
   v_new_salary    NUMBER(15, 2);
   v_total_bonus   NUMBER(15, 2) := 0;
   v_count        NUMBER := 0;

BEGIN

DBMS_OUTPUT.PUT_LINE('================================================
=');
   DBMS_OUTPUT.PUT_LINE('  Scenario 2: Employee Bonus Update            ');
   DBMS_OUTPUT.PUT_LINE('  Department  : ' || p_department);
   DBMS_OUTPUT.PUT_LINE('  Bonus       : ' || p_bonus_percent || '%');

DBMS_OUTPUT.PUT_LINE('================================================
=');

   -- Validate bonus percentage
   IF p_bonus_percent <= 0 OR p_bonus_percent > 100 THEN
      DBMS_OUTPUT.PUT_LINE('ERROR: Bonus percentage must be between 1 and
100.');
      RETURN;
```

```
  END IF;

  FOR rec IN cur_dept_employees LOOP

    v_bonus      := ROUND(rec.Salary * (p_bonus_percent / 100), 2);
    v_new_salary := rec.Salary + v_bonus;

    UPDATE Employees
    SET    Salary = v_new_salary
    WHERE  EmployeeID = rec.EmployeeID;

    v_total_bonus := v_total_bonus + v_bonus;
    v_count       := v_count + 1;

    DBMS_OUTPUT.PUT_LINE(
      'Employee: ' || rec.EmployeeName      ||
      ' | Old Salary: Rs.' || rec.Salary     ||
      ' | Bonus: Rs.' || v_bonus           ||
      ' | New Salary: Rs.' || v_new_salary
    );

  END LOOP;

  IF v_count = 0 THEN
    DBMS_OUTPUT.PUT_LINE('No employees found in department: ' ||
p_department);
  ELSE
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('-----------------------------------------------');
    DBMS_OUTPUT.PUT_LINE('Employees updated  : ' || v_count);
    DBMS_OUTPUT.PUT_LINE('Total bonus paid   : Rs.' || v_total_bonus);
    DBMS_OUTPUT.PUT_LINE('Status : Bonus updated successfully.');

DBMS_OUTPUT.PUT_LINE('===============================================
=');
  END IF;

EXCEPTION
  WHEN OTHERS THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);
END UpdateEmployeeBonus;
```

OUTPUT:

```
Procedure created.
```

SCENERIO 3:

CODE:

```
CREATE OR REPLACE PROCEDURE TransferFunds (
   p_from_account  IN NUMBER,
   p_to_account    IN NUMBER,
   p_amount        IN NUMBER
) AS

   v_from_balance  NUMBER(15, 2);
   v_to_balance    NUMBER(15, 2);
   v_from_name     VARCHAR2(100);
   v_to_name       VARCHAR2(100);

BEGIN

DBMS_OUTPUT.PUT_LINE('================================================
=');
   DBMS_OUTPUT.PUT_LINE('  Scenario 3: Fund Transfer                   ');

DBMS_OUTPUT.PUT_LINE('================================================
=');

   -- Validate amount
   IF p_amount <= 0 THEN
      DBMS_OUTPUT.PUT_LINE('ERROR: Transfer amount must be greater than 0.');
      RETURN;
   END IF;

   -- Fetch source account details
   BEGIN
      SELECT Balance, CustomerName
      INTO   v_from_balance, v_from_name
      FROM   Accounts
      WHERE  AccountID = p_from_account;
   EXCEPTION
      WHEN NO_DATA_FOUND THEN
         DBMS_OUTPUT.PUT_LINE('ERROR: Source account ' || p_from_account || '
not found.');
         RETURN;
   END;

   -- Fetch destination account details
   BEGIN
      SELECT Balance, CustomerName
```

```
      INTO   v_to_balance, v_to_name
      FROM   Accounts
      WHERE  AccountID = p_to_account;
   EXCEPTION
      WHEN NO_DATA_FOUND THEN
         DBMS_OUTPUT.PUT_LINE('ERROR: Destination account ' || p_to_account || '
not found.');
         RETURN;
   END;

   DBMS_OUTPUT.PUT_LINE('From  : ' || v_from_name || ' (ID: ' || p_from_account
|| ')');
   DBMS_OUTPUT.PUT_LINE('To    : ' || v_to_name   || ' (ID: ' || p_to_account   ||
')');
   DBMS_OUTPUT.PUT_LINE('Amount: Rs.' || p_amount);
   DBMS_OUTPUT.PUT_LINE('-----------------------------------------------');

   -- Check sufficient balance
   IF v_from_balance < p_amount THEN
      DBMS_OUTPUT.PUT_LINE('ERROR: Insufficient balance!');
      DBMS_OUTPUT.PUT_LINE('Available: Rs.' || v_from_balance ||
                           ' | Required: Rs.' || p_amount);

      -- Log failed transaction
      INSERT INTO TransactionLog
      VALUES (txn_seq.NEXTVAL, p_from_account, p_to_account,
            p_amount, SYSDATE, 'FAILED');
      COMMIT;
      RETURN;
   END IF;

   -- Deduct from source
   UPDATE Accounts
   SET    Balance = Balance - p_amount
   WHERE  AccountID = p_from_account;

   -- Credit to destination
   UPDATE Accounts
   SET    Balance = Balance + p_amount
   WHERE  AccountID = p_to_account;

   -- Log successful transaction
   INSERT INTO TransactionLog
   VALUES (txn_seq.NEXTVAL, p_from_account, p_to_account,
         p_amount, SYSDATE, 'SUCCESS');

   COMMIT;
```

```
   DBMS_OUTPUT.PUT_LINE('Transfer Successful!');
   DBMS_OUTPUT.PUT_LINE(v_from_name || ' new balance : Rs.' ||
(v_from_balance - p_amount));
   DBMS_OUTPUT.PUT_LINE(v_to_name   || ' new balance : Rs.' || (v_to_balance   +
p_amount));
   DBMS_OUTPUT.PUT_LINE('Status : Fund transfer completed successfully.');

DBMS_OUTPUT.PUT_LINE('==============================================
=');

EXCEPTION
   WHEN OTHERS THEN
      ROLLBACK;
      DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);
END TransferFunds;
```

EXECUTION :

```
SQL> EXEC TransferFunds(1001, 1002, 5000);
==============================================
Scenario 3: Fund Transfer
==============================================
From   : Rajesh Kumar (ID: 1001)
To     : Priya Sharma (ID: 1002)
Amount: Rs.5000
----------------------------------------------
Transfer Successful!
Rajesh Kumar new balance : Rs.45000
Priya Sharma new balance : Rs.35000
Status : Fund transfer completed successfully.
==============================================

PL/SQL procedure successfully completed.

SQL> EXEC TransferFunds(1005, 1001, 50000);
==============================================
Scenario 3: Fund Transfer
==============================================
From   : Vikram Nair (ID: 1005)
To     : Rajesh Kumar (ID: 1001)
Amount: Rs.50000
----------------------------------------------
ERROR: Insufficient balance!
Available: Rs.15000 | Required: Rs.50000

PL/SQL procedure successfully completed.
```